

Parameterized UART Transceiver in Verilog

Izaiah Thigpen

RTL Design & Verification · Verilog · Icarus Verilog / Xilinx Vivado

Overview

A UART (Universal Asynchronous Receiver/Transmitter) is the hardware block that lets a chip exchange data over a simple two-wire serial link — the backbone of interfaces such as RS-232, Bluetooth modules, GPS receivers, and countless sensor and debug ports. This project implements a complete UART in Verilog following the standard **8-N-1** framing (8 data bits, no parity, 1 stop bit). Rather than hard-coding a single baud rate, the design is **parameterized** by system-clock frequency, baud rate, and oversampling ratio, so the same source retargets to any FPGA or clock simply by changing parameters. The full design is synthesizable and simulates on free tooling (Icarus Verilog) as well as industry tools (Xilinx Vivado).

How a UART Frame Works

The line idles high; a transmission begins with a **start bit** (high-to-low), followed by eight data bits sent **LSB-first**, and closes with a **stop bit** (high). Because transmitter and receiver run on independent clocks with no shared strobe, the receiver must recover timing from the data itself — the central challenge this design solves.

Architecture

- **Baud generator** — a rounded clock divider that emits a tick strobe at 16× the baud rate. This single time base is shared by both the transmit and receive paths, and the rounding keeps the baud error well under 1% for typical clock/baud combinations.
- **Transmitter** — a four-state machine (IDLE → START → DATA → STOP) that latches a byte, drives the start bit, shifts the eight data bits out LSB-first while holding each for one bit period, then drives the stop bit and raises a busy flag for the duration of the frame.
- **Receiver** — the interesting part. An asynchronous input is first passed through a **two-flop synchronizer** to avoid metastability. A **16× oversampling** FSM then watches for the start-bit edge, waits half a bit period to re-center on the middle of the start bit, and from there samples each data bit at its center. To reject glitches, each bit is decided by a **3-sample majority vote** taken around the bit center. A **framing-error** flag is raised if the stop bit is not high, catching baud mismatches or line faults.

Verification

The design is exercised by a **self-checking testbench** that wires the transmitter output straight back into the receiver (a loopback), so every byte transmitted is immediately decoded and compared against the original. Mismatches and framing errors increment an error counter, and the run ends by printing an automatic PASS / FAIL verdict — no manual waveform inspection required. Two testbenches are provided:

Testbench	Bytes sent	Purpose
testbench.sv (full)	512: all 256 byte values + 256 pseudo-random	Exhaustive regression — proves every possible byte round-trips correctly. Produces a large waveform, so it is meant for the PASS/FAIL log, not for viewing.
testbench_demo.sv (short)	5: "H", "I", "!", 0xA5, 0x3C	Quick, human-readable demo whose waveform fits on one screen — used to capture Figure 1 and to eyeball the protocol.

Results

The full testbench passed all **512 bytes with 0 errors**. To bound real-world robustness, the FSM was additionally cross-checked against an independent bit-accurate reference model, which confirmed correct decoding under a **±3% baud-rate mismatch** between transmitter and receiver — comfortably beyond the tolerance an 8-N-1 link requires. Figure 1 shows the short demo decoding five bytes; each value appears on rx_data exactly as rx_valid pulses.



Figure 1. Simulation waveform (EPWave): five bytes decoded on rx_data — 0x48, 0x49, 0x21, 0xA5, 0x3C ("H", "I", "!", ...) — each latched when rx_valid pulses.

Tools & Skills Demonstrated

Verilog RTL design; finite-state-machine modeling; asynchronous clock-domain crossing and metastability handling; oversampling and majority-vote data recovery; parameterized/reusable IP; self-checking testbench methodology and functional simulation (Icarus Verilog, Xilinx Vivado); VCD waveform analysis.